



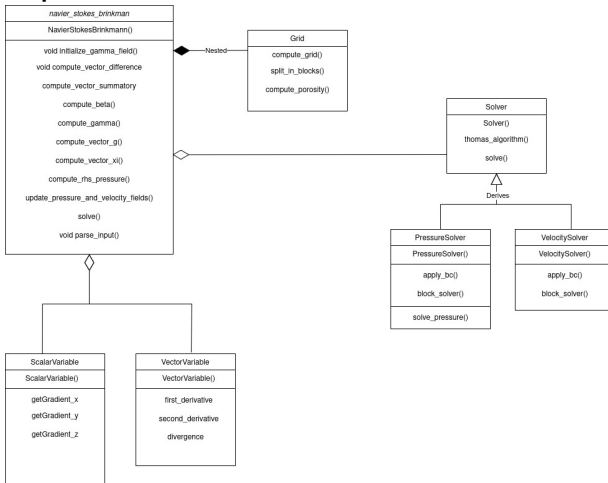
POLITECNICO
MILANO 1863

High Performance Scientific Computing for Aerospace Project

10th November 2025 update

November 10th, 2025

■ Classes composition:



Structure of the input file:

- **Mesh points:** Number of points in X, Y, Z directions
 - ▶ Example: 50 50 50 $\Rightarrow N_x = 50, N_y = 50, N_z = 50$
- **Time parameters:**
 - ▶ dt - Time step size, e.g., 0.01
 - ▶ T - Total simulation time, e.g., 10.0
- **Spatial step sizes:**
 - ▶ dx dy dz - Grid spacing in X, Y, Z, e.g., 0.5 0.5 0.5
- **Initial conditions files:**
 - ▶ u0_init_file - File with initial velocity values
 - ▶ p0_init_file - File with initial pressure values
 - ▶ k_file - File with permeability values

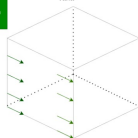
Boundary conditions analysis

3/6

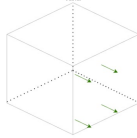


This x component does not need computation

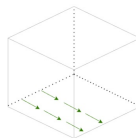
$y=0$
 x/z
varia



$x=1$
 y/z
varia

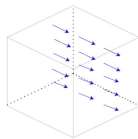


$z=0$
 x/y
varia

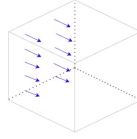


This x components needs computation

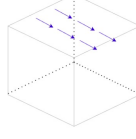
$y=1$
 x/z
varia



$x=0$
 y/z
varia



$z=1$
 x/y
varia



Velocity Boundary Conditions

4/6

For each of the three steps of the algorithm for momentum equation, the Dirichlet Boundary Conditions are applied in the following way:
when solving for the component parallel to the decomposition direction:

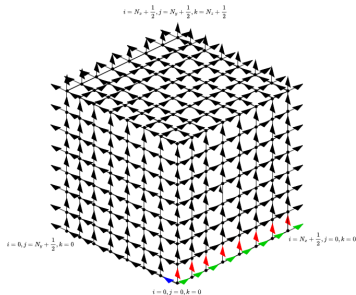
$$\begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ a & b & c & \ddots & \vdots \\ 0 & a & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & b & c \\ 0 & \cdots & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \eta_{1,x} \\ \eta_{2,x} \\ \vdots \\ \eta_{N-1,x} \\ \eta_{N,x} \end{bmatrix} = \begin{bmatrix} u_{0,j,k} + \frac{\partial u}{\partial x} \Big|_{(0,j,k)} \frac{\Delta x}{2} \\ r_{2,x} \\ \vdots \\ r_{N-1,x} \\ \text{BC value} \end{bmatrix}$$

when solving for a component perpendicular to the decomposition direction:

$$\begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ a & b & c & \ddots & \vdots \\ 0 & a & \ddots & \ddots & 0 \\ \vdots & \ddots & a & b & c \\ 0 & \cdots & 0 & a & b - c \end{bmatrix} \begin{bmatrix} \eta_{1,y} \\ \eta_{2,y} \\ \vdots \\ \eta_{N-1,y} \\ \eta_{N,y} \end{bmatrix} = \begin{bmatrix} \text{BC value} \\ r_{2,y} \\ \vdots \\ r_{N-1,y} \\ r_{N,y} - 2cu_{ex} \end{bmatrix}$$

We found a structural pattern in our staggered grid that simplifies boundary condition (BC) logic. Let x_0 be the solving direction and x_1 and x_2 , with indices i_1 and i_2 , the first and second direction along which the solver iterates. Then:

- **Normal Component:** i_1 is **even** AND i_2 is **even**.
- **Tangential Component:** i_1 is **odd** OR i_2 is **odd**.



This pattern holds for all **three velocity components** and allows for a **systematic implementation** of BCs. The logic is implemented in the code simply by modifying the **stride computation** for each direction, a task easily programmed with templates.

Code Validation

- Compute error
- Compute residual
- Convergence study

Parallelization Strategy

- Vectorization of the sequential code
- Parallelization of the code